

conventional one we all learned in school (see Figure 2). Lots of optimised algorithms exist, but let's leave that for another article.

System/software information

- System: SGI Altix UV1000 (128-core 16-CPU Node 1 TB RAM Fat-Tree model cc-NUMA)
- OS: RHEL 6.1 x86_64
 - GCC version: Red Hat 4.4.5-6

To reduce the required space, the source code is provided separately. Download it (`psmm.c`) from http://www.linuxforu.com/article_source_code/june12/parallel_matrix_code.zip.

Code compilation and output

I compiled the code with GCC optimisation level 2. The sample code output shown below is produced for 128 threads, matrix size 3000 and chunk size 1:

```
server1 [root]> gcc -O2 omp_mmul_svp.c -o omp_mmul -fopenmp
server1 [root]> ./omp_mmul 128 3000 1
Multiply Array memory allocation...
Assigning array elements with '1' for multiply op...
Matrix multiplication with openmp...
Chunk Size = 1
Total worker threads invoked = 128
[+]multiply_op_omp_time 4.379983 sec
Single thread verification complete...OK
```

Thread monitoring

To monitor threads on the system, start the `top` utility with the `Threads` toggle option enabled, i.e., `top -H`. Threads are

part of the same process. Using the P column in `top`, we can determine the core used for processing the thread.

Code information

Frequently used shared and thread-private variables are initialised as register variables, directing the compiler to keep them in CPU registers for faster access. The number

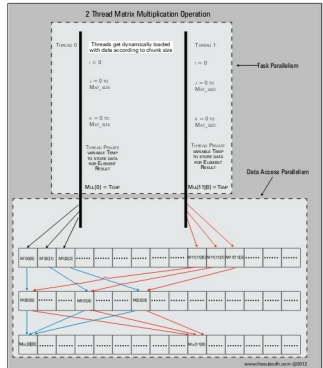


Figure 2: Two-thread matrix multiplication flow

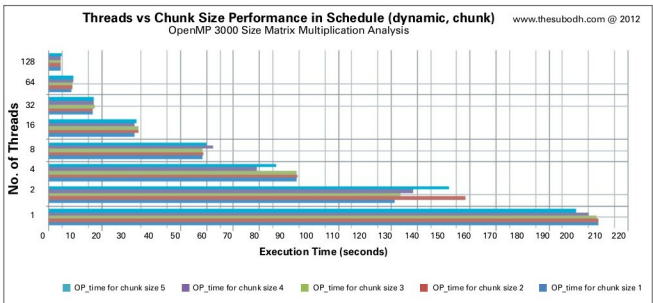


Figure 3: Evaluating the performance of threads vs chunk size